

TAIKO · PROTOCOL DESIGN · AUGUST 2026

# Based Preconfirmation Redesign

A perpetual auction with commit → publish → seal epochs  
Design v8 — a learning deck

Anyone can win the sequencing seat

Slashing from day one

Finality at the seal

No oracles

## MOTIVATION

# Why redesign preconfirmations?

### Where we are today

- Preconfirmations run on a **trusted whitelist** of operators — users trust reputation, not collateral.
- The planned permissionless path required preconfers to be **L1 validators** registered in a Universal Registry Contract (URC) — blocked twice over: little validator adoption, heavy integration burden.
- Slashing machinery exists on paper but is **dormant**: bonds are configured to zero, faults have no teeth.
- A preconf promise today is only as good as the operator's brand.

### The redesign's premise

- Decouple the seat from L1 validators: sell sequencing rights in an **open, perpetual on-chain auction**.
- Make every promise **bonded and objectively slashable** from the first day.
- Make finalization **atomic with proving** — the proposal itself carries a validity proof, so there is no separate prover market and no finality lag.
- Design the **failure modes first**: every degraded state has a permissionless exit.

## MOTIVATION

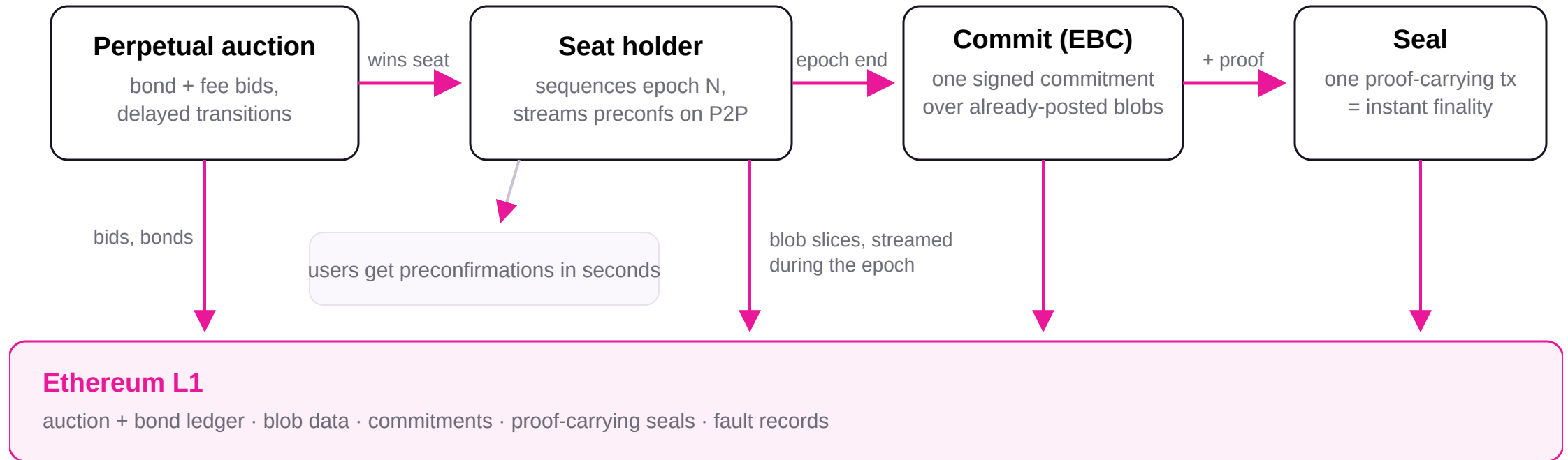
# Five goals steer every mechanism

| GOAL      | WHAT IT DEMANDS                                                                                                                                                                |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>G1</b> | <b>Anyone can become the preconfer</b> — a perpetual auction for a single sequencing seat, open to any bonded bidder (no validator requirement).                               |
| <b>G2</b> | <b>Slashing from day one</b> — on objective faults only: liability is computed from L1 records, never from someone's report.                                                   |
| <b>G3</b> | <b>Non-validators can act reliably on L1</b> — every mandatory action has a multi-slot budget; the seal has a multi-epoch window.                                              |
| <b>G4</b> | <b>Sealed epochs finalize immediately</b> — proof verified at acceptance; fast withdrawals; no separate prover market.                                                         |
| <b>G5</b> | <b>Explicit degradation endgames</b> — the chain keeps moving through every failure; in the permissionless phase anyone can end a degradation by outbidding the reserve floor. |

G5 is a *liveness* guarantee, not a user-fund-safety guarantee: an L1 reorg deeper than the grace  $\kappa$  can revert L2 state exactly as on any rollup.

## MENTAL MODEL

# The big picture



One seat at a time; one open epoch at a time; every promise backed by bond; every outcome provable by anyone from L1 data.

# Roles and tenures

- **Seat holder** — the current auction winner; sequences epochs, issues precons, commits and seals them.
- **Standby bidders** — bonded runners-up; auto-promoted (bindingly) if the holder quits, idles, or is slashed out.
- **Observers** — anyone; earn bounties for recording faults and recovering stalled epochs.
- **Users** — receive preconfirmations; can always force-include through L1.
- **Treasury / DAO** — receives auction fees; governs parameters prospectively.
- **L2 nodes** — re-derive everything from L1; hold nobody's word for anything.

## Tenure = the accountable identity

A tenure is an immutable id binding the holder, its registered keys, its assigned epochs, and its reserved collateral. Every artifact it signs binds (chain, tenure, epoch, role).

Every fault gets a stable logical id —  $H(\text{chain, tenure, epoch, class, position})$  — debited **at most once**, ever.

## Two-phase rollout

**Phase A:** auction entry temporarily allowlisted (training wheels; DAO commits to a fast-path SLA). **Phase B:** fully permissionless. Same mechanics in both.

# The perpetual auction (on L1)

- One sequencing seat, auctioned **perpetually** — the incumbent keeps it until outbid, quitting, or terminated.
- A bid = **TAIKO liveness bond** + **ETH deposit** + **per-epoch ETH fee** paid to the treasury; a new bid must beat the incumbent by  $\geq 10\%$ . The ETH lives in **one account with a seniority waterfall**: recovery tranche → safety tranche → working tranche, under a single admission solvency check.
- Every transition is delayed by  $q = 2$  epochs — the current and next epochs are always final, so handovers are never a surprise.
- Exits are never free: quitting gives  $q$  epochs of notice; **idling pays exactly what quitting would** (the fee clock keeps running), so going dark is never the cheaper exit.
- Termination on: a settled fault, bond below reservation, or  $K_{\text{empty}} = 16$  consecutive epochs serving nobody but yourself.

## Why the auction lives on L1

The auction and the bond ledger must keep operating even when the L2 itself is degraded or halted — so they live on L1. Slashing *evidence* is proven on L2 (cheap, expressive) and the verdict is bridged up to the single L1 ledger.

## No perpetual grandfathering

Every *new* epoch assignment — the incumbent's included — is gated on the *current* reservation. Raise the floor, and a holder that does not top up is gracefully rotated out.

# A split bond — and no oracles, anywhere

## Safety bond — in ETH

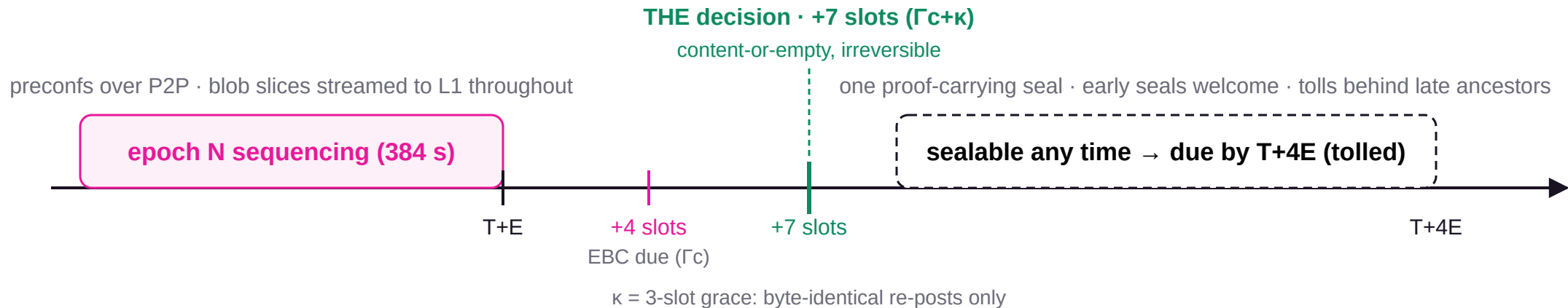
The safety tranche of the tenure's single ETH account. Backs the **theft** class: equivocation / MEV theft, where the attacker's gain is ETH-denominated. Deterrence and gain share the same numéraire, so it is sized to exceed per-epoch extractable MEV **without any price feed** — and its value is fixed from tenure start.

## Liveness bond — in TAIKO

Backs the **delay** class: missed commits and seals, which slow the chain but steal nothing. A TAIKO price move weakens grieving deterrence at worst — a bounded, governance-adjustable residual.

- **Owner directives honored together:** TAIKO as the bond token, and no price oracles — ETH is assumed stable-to-increasing.
- A TAIKO price crash cannot cheapen the theft slash; the short-then-misbehave and withdraw-after-shock attacks disappear because the theft debit's value never re-prices.
- Liveness slashes are  $\geq 80\%$  **burned** — faults must destroy value, not move it.
- All payouts the protocol ever makes go to **precommitted payees** (proof public inputs or pre-registrations) — front-running a copied proof or poke redirects nothing.

# One epoch's life on the clock



While epoch N settles, epoch N+1 is already sequencing: its first ~7 slots are **soft** preconfs (parent-provisional), upgrading to **hard** (bond-backed) the moment N's outcome is final. Proving latency (~10–15 min zkVM) fits well inside the 4-epoch seal deadline.



# The EBC — one commitment decides the epoch

- The **epoch-boundary commitment** is one-shot per (tenure, epoch) — and **presence at the single decision (+7 slots) is all that counts**: a first submission any time in those 7 slots is valid, so the honest budget is the full span, not just the 4-slot nominal deadline.
- It binds the full ordered content, the end-of-preconf tip, and the **hashes of blob slices already posted to L1** — data first, commitment second.
- It also commits **its own L1 origin** (anchor block hash/state root). Derivation reads the committed origin, never the block that happened to include the transaction — so an L1 reorg can shift *when* things land, never *what* they mean.
- The committed anchor must be  $\geq 32$  slots deep (reorg safety), fresh, and advancing (no serving from stale L1 state).
- Miss the EBC  $\Rightarrow$  the epoch resolves **EMPTY** and a liveness certificate settles. An explicit empty EBC is valid — unless forced inclusions are pending.

## **Totality: garbage cannot wedge the chain**

Derivation is **total and bounded**: any committed bytes map to exactly one L2 block sequence. Malformed or oversized content degrades *deterministically* to default content — under consensus-exact parse caps (size, RLP count/depth, tx count) enforced identically by client and proof circuit.

Consequence: a committed epoch has **one canonical outcome, provable by anyone holding the data**, no matter when or where anything landed on L1.

## Publication is part of the commitment

- There is **no separate publication deadline**: the holder streams its blob data to L1 *during its own epoch*, and the EBC is valid **only if the data it references is already there**.
- So content-or-empty is a **single decision, 7 slots after the epoch** — not a second judgment ~27 slots later.
- The  $\kappa$  grace exists for exactly one thing: if a referenced slice was reorged out, **anyone** may re-post the byte-identical slice. Nothing new can ever be introduced.
- If a third party fills a slice the holder failed to keep available, the filler earns a reward — **funded by the failing holder**, paid to a pre-registered beneficiary. No fault, no reward, no honest party to front-run.

### Why this kills the "last look"

A predecessor that published on time has **no residual choice left** — its outcome is locked by its own EBC. The successor's exposure to a mischievous parent shrinks to the fixed 7-slot soft-preconf window (~22% of an epoch), down from ~84% under a late-publication design.

### Censorship has to beat everyone

Suppressing an epoch means censoring blob posts across a 32-slot span *plus* the re-post grace, against **every holder of the data** — the most expensive artifact to censor, priced explicitly in the game-theory analysis.

## The seal: proving is finalizing

- One small, retryable L1 transaction carries the validity proof for the epoch's canonical outcome; **acceptance = finality** (G4).
- Proofs are fully **precomputable**: content is final 7 slots after the epoch, and the proof binds no later L1 state. The seal has a **deadline only** (due by T+4E, tolled) — sealing early is always valid and shrinks everyone's exposure.
- The seal contributes **zero derivation inputs** — sealing late or from a different sender can never change what the epoch means (I7).
- The payout address is a **public input of the proof**: copying a proof from the mempool advances the chain but pays the original prover's payee (I8).

### One open epoch

Epochs seal **strictly in order** through a single openEpoch pointer. Every later deadline *tolls* while an ancestor is unresolved — a slow predecessor never converts into a successor's fault.

### The recovery lane never closes

At every moment some permissionless action can advance openEpoch: the content seal, a forced-only seal, an empty seal, or — as an absolute floor — the bounded cancellation path. Unproven material never blocks any of them (I3).

# What a preconfirmation means now

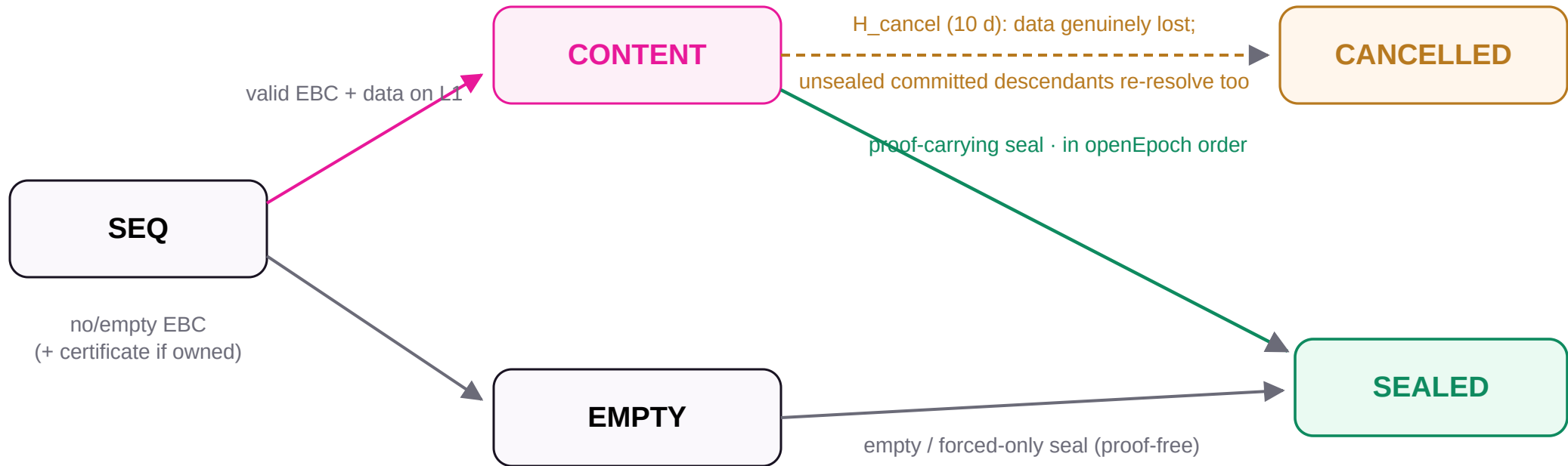
| MOMENT                                  | THE PROMISE YOU HOLD                                                                                      | BACKED BY                            |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------|
| <b>Seconds</b> — P2P preconf            | A signed commitment binding (tenure, epoch, position, hashes, deadline) — equivocation is a safety fault. | ETH safety bond (MEV-sized)          |
| <b>First ~7 slots of the next epoch</b> | Soft preconf: explicitly provisional on the parent's single decision.                                     | Parent's own EBC + bond              |
| <b>7 slots after the epoch</b>          | Content-or-empty is <b>irreversible</b> ; hard preconfs from here.                                        | Protocol invariant (single decision) |
| <b>At the seal</b>                      | Proven, finalized L2 state on L1.                                                                         | Validity proof                       |

Force-included transactions ride the same rails: once snapshotted, an epoch's minimum valid outcome *includes them* — an empty resolution is simply invalid while the forced queue is non-empty (l6).

# Nine invariants everything must obey

- **I1 — Total, bounded, content-addressed derivation.** Committed bytes have one canonical outcome, independent of all L1 timing.
- **I2 — No fault requires the accused.** Liability is computed state, matured by deadline + absence; every reader materializes it. Pokes accelerate, never gate.
- **I3 — Single open epoch, always advanceable.** A permissionless advancing action always exists.
- **I4 — Successor-safe parent.** Outcomes irreversible at  $\Gamma c + \kappa$ ; deadlines toll behind blocked ancestors.
- **I5 — Bounded global backlog.**  $\text{Lag} > K \Rightarrow$  recovery-only mode; the unsealed content tail is structurally capped.
- **I6 — Forced inclusions are censorship-proof.** Non-empty forced queue  $\Rightarrow$  empty outcomes invalid; the forced-only epoch is constructible by anyone.
- **I7 — Only proven transitions lock state; outcomes are sender-free.**
- **I8 — Payees are precommitted, not first-claimers.** Every reward is bound before the witness is public.
- **I9 — Deterrence value is oracle-free.** ETH-matched slashes are ETH-denominated; no mechanism reads a price.
- **Corollary — sealed epochs are final.** No mechanism, including disaster cancellation, ever touches sealed state.

# The epoch state machine



The decision (7 slots) picks **CONTENT** or **EMPTY** **once**; seals close epochs **strictly in openEpoch order**; sealed state is untouchable, forever.

The disaster path is the only exception route: cancelling a stuck epoch marks every unsealed descendant commitment that chained to it VOID (each then closes as cancelled, in order), re-queues their forced items at the front, and bills the tenure that caused it — sealed history is never touched.

## Forced inclusion: the floor nobody can lower

- Users enqueue transactions on L1 with a fee  $a + b \cdot \text{bytes} + c \cdot \text{declared\_gas}$  — L1-measurable quantities with conservative constants that upper-bound proving cost; the per-item base makes flooding pay per item.
- Each epoch takes a deterministic **snapshot** of the queue head; overflow spills to the next epoch.
- While the snapshot is non-empty, the epoch's minimum valid outcome is the **forced-only epoch** — constructible and provable by *anyone* from L1 data alone. Empty is invalid (I6).
- Fees flow to whoever seals the consuming epoch — the fallback funds its own proving.

### One lifecycle, no double-spends

Every forced item advances queued → snapshot → consumed | refunded on L1, with its payer stored at enqueue time. Seal proofs take the item's status as a public input; a refund atomically kills every live commitment containing the item. Refund *and* execution can never both happen.

### Sybil-proof accounting

Self-inclusions don't reset the idle counter ( $K_{\text{empty}}$  counts epochs with no genuine third-party content), forced-only fee income per tenure is capped, and a holder sealing its own epochs earns no recovery bounty. Squatting has no profit path.

# Faults are computed, never reported

- A liveness fault is an **objective L1 fact**: assignment + deadline + absence. It **matures** the instant its deadline-plus-grace passes — no transaction needed, no cooperation from the accused.
- Anyone may record ("poke") a fault for a bounty, but pokes only *accelerate*: every function that depends on fault status — withdrawal, sealing an empty outcome, promotion — **computes maturity itself and materializes the certificate as part of the call**.
- Consequence: censoring pokes buys nothing. A holder that missed an obligation *faults itself by trying to withdraw*.
- Each deadline has exactly **one  $\kappa$  grace** and **one atomic transition**: present  $\Rightarrow$  accepted; absent  $\Rightarrow$  certificate settled, parent outcome fixed, slashability locked — all at the same instant.

## Content faults: evidence on L2, money on L1

Equivocation and content faults are proven cheaply on L2 by permissionless observers; the verdict is bridged to the **single L1 bond ledger**, which alone enforces one-debit-per-fault. Verdicts bind a finalized origin root + incarnation number, so a verdict minted on an orphaned fork is dead on arrival after a deep reorg.

## Withdrawal gate

Exit requires: zero unresolved certificates (computed, not poked), no unsealed assigned epochs, no in-flight verdicts, then a  $\geq 2$ -week floor. Bonds outlive every obligation they back.



# When a holder fails, the chain barely notices

## The failure playbook

- **Missed commit** → epoch resolves EMPTY at the single decision; certificate settles; slash debits; chain continues.
- **Missed seal** → the epoch's data is on L1, so from the moment the deadline fault matures **anyone** seals it — paid at indexed cost *from the fault's own recovery tranche*, never from honest tenures.
- **Termination** → the highest standby is auto-promoted (bindingly, after the q-delay); no re-auction gap. No standby ⇒ Total Anarchy (next slides).
- **Every recovery is compensated, never framed:** certificates name only the tenure that actually held the duty; deadlines toll behind blocked ancestors.

## Solvent by admission

The senior tranche of each tenure's single ETH account is sized to its worst-case K outstanding recovery obligations, checked at admission:  $\Sigma \text{ reserves} \geq \Sigma \text{ worst-case obligations}$ . A deliberate fault can only ever spend its own account; a thin shared pool covers only the honest remainder.

## Recovery pays cost, not prizes

Compensation is a capped multiple of prevailing L1 + proving costs (indexed, not fixed) — gas spikes raise it, and there is no margin worth attacking for. Deterrence lives in the  $\geq 80\%$  burn, not the bounty.

# A ladder, not a kill switch

## Rung 1 · The seal deadline — fault-paid resolution, nothing global

The moment a stuck epoch's seal deadline passes, its fault matures and anyone seals it from L1 data, paid from the faulters' recovery tranche. One bad holder is never anyone else's problem — and no separate horizon is needed.

## Rung 2 · Recovery-only mode — when lag exceeds $K = 8$ epochs

New discretionary content pauses; all effort funnels into the recovery lane. No withdrawal freeze — honest tenures with clean books may still exit.

## Rung 3 · Attested systemic-outage freeze — never reachable by one holder

A global pause of maturation + withdrawals requires an independent attestation of a genuine proof-system outage — never openEpoch age alone. Forgiveness never covers whoever drove the stall.

Absolute floor:  $H_{cancel}$  (10 days, data genuinely unavailable) — the epoch re-resolves forced-only/empty, unsealed descendants re-resolve deterministically, and the causing tenure pays for the whole cascade. Sealed history is never touched.

## Total Anarchy — the fallback with an exit sign

- No holder and no standby  $\Rightarrow$  epochs are unowned: they resolve empty or **forced-only**, sealed permissionlessly — the bridge and forced lane keep flowing at L1 cadence.
- No discretionary content, no precons, no slashing — nothing is bonded to promise anything. (A v7 attempt to restore FCFS content via atomic proposals was **reverted by the v8 regression audit**: a cheap empty seal beats any ~15-minute proof, re-opening a censorship lever an earlier review round had closed.)
- **Phase B exit**: anyone ends anarchy by bidding the reserve floor — a censorship-resistant fallback, not a sequencing mode.
- **Phase A exit**: the DAO's pre-committed fast-path SLA expands the allowlist within a published deadline.

### Deposits stay safe, and bounded

Even a worst-case voided forced item settles as a refund: queue fees refundable on L1, bridged messages refundable through the bridge's unprocessed-message path. Worst-case bridge settlement is **H\_cancel + one forced-only cadence** — a number a depositor can price, not an open-ended limbo. Value is only ever delayed, never burned.

### Why anarchy is acceptable

It is the *floor*, not the plan: reaching it requires the seat, every standby, and every would-be bidder to decline a profitable seat — and staying in it is exactly as hard as out-bidding a floor price.

# Attacks the design prices out

| ATTACK                                       | WHY IT FAILS                                                                                                                                   |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Preconf equivocation / MEV theft</b>      | ETH safety bond sized above per-epoch extractable MEV; value fixed from tenure start — no oracle, no shorting window, no price-crash discount. |
| <b>Squat the seat, serve nobody</b>          | Idling pays quit-equivalent fees; K_empty terminates; self-traffic doesn't reset the counter; forced-only income capped.                       |
| <b>Withhold the seal, hold users hostage</b> | Data is already on L1; the deadline's matured fault lets anyone seal at the faulters expense. A stall buys ~minutes, and costs a slash.        |
| <b>Censor the poke, then exit</b>            | Liability is computed at read time — withdrawal itself materializes the fault (I2).                                                            |
| <b>Front-run proofs / pokes / fills</b>      | Payees are precommitted (proof public inputs, pre-registrations) — copying a witness pays the original beneficiary (I8).                       |
| <b>Flood forced inclusions</b>               | Per-item base fee + byte/gas-proportional pricing bounds proving cost; snapshots cap per-epoch load, overflow spills deterministically.        |
| <b>Publish a long tail, then cancel it</b>   | Recovery-only mode caps the unsealed content tail at $\approx K+d+s$ epochs (~77 min) — a 14-day cancellation avalanche cannot exist.          |

Stated residuals: sub-H\_force targeted censorship can cost an honest holder one liveness slash (priced unprofitable); TAIKO-price erosion weakens only grieving deterrence; K\_empty bounds self-serving, but a complete censorship bound awaits the max-tenure-duration decision.

# How hard has this been attacked already?

## 10 passes

of adversarial review across 6 rounds (multiple independent AI reviewers), every finding accepted-or-rebutted in a public disposition table. An owner-approved simplification round (v7) was then **re-audited against every prior finding** — the one change that regressed an old fix was reverted (v8).

## 4.4 M states

explored by an exhaustive model checker of the epoch state machine across three bounds — **zero invariant violations, zero permanent halts**, with the adversary free to interleave every legal action.

### The checker checks itself

8 deliberately-injected design bugs (double decision, ignore-forced, out-of-order seal, ungated withdrawal, missing cascade, double debit, evidence erasure, unmaterialized fault) — **8/8 caught** by the invariant written to catch each.

| BOUND | STATES    | VIOLATIONS | HALTS |
|-------|-----------|------------|-------|
| 3 × 4 | 620,966   | 0          | 0     |
| 3 × 5 | 1,025,874 | 0          | 0     |
| 4 × 3 | 2,722,729 | 0          | 0     |

Bounded checking of a logical abstraction — strong evidence for the state machine, not a proof of the eventual Solidity.

# Status and reading map

## Before implementation (§13-S)

- Deep-reorg ( $> \kappa$ ) joint-rewind semantics; cascade determinism proof.
- Consensus-exact parse caps across client and circuit; derivation-input sign-off (Appendix C).
- Record-spine retention/compaction; recovery-tranche sizing; forced-item nullifier; verdict incarnation; precommitted-payee binding.

## Tuning before Phase B (§13-T)

- Soft-window width vs duty-cycle; max tenure duration (the complete censorship bound);  $K/K'/K\_empty/H\_cancel$  calibration; Phase A  $\rightarrow$  B criteria.

## Read the full story

`docs/preconfirmation/README.md` — index

`status-quo.md` — what exists today, premise validation

`redesign-proposal.md` — the v6 design (this deck's source of truth)

`simulation/` — model checker + results

`reference/` — the two prior Notion design docs, converted

## Discuss

PR **taikoxyz/taiko-mono #22034** — all review rounds, defenses, and dispositions are on the thread and in Appendix B.